

Simulation de fluides en 2D

Projet de programmation 2

Axel Pereyrol, Saurane Delrieu, Gaelle Milezi, Satine Barraux

L3 informatique

Faculté des Sciences

Université de Montpellier

2025-2026

Résumé

Ce projet de simulation de fluides en 2D consiste à rendre de manière interactive et en temps réel l'évolution d'un fluide dans une grille. Nous avons implémenté la méthode de Jos Stam "Stable Fluids" pour simuler le fluide, et nous avons développé une interface graphique pour visualiser et interagir avec la simulation. L'utilisateur peut ajouter des sources de fluide, créer des obstacles, et modifier les paramètres du fluide en temps réel.

Table des matières

1	Présentation du Sujet	3
2	Développements Logiciel : Conception, Modélisation, Implémentation	3
2.1	Technologies utilisées	3
2.2	Développements logiciel réalisés	4
2.2.1	Affichage d'une grille de chaleur	5
2.2.2	Simulation du fluide	7
2.2.3	Mode point/ligne	7
2.2.4	Gestion de la chaleur	8
2.2.5	Gestion des obstacles	10
2.2.6	Modules en cours de développement	13
2.3	Interface utilisateur	14
2.4	Format des données d'entrée	14
2.5	Statistiques	14
3	Algorithmes et Structures de Données	15
3.1	Principaux algorithmes utilisés	15
3.1.1	Advection	15
3.1.2	Diffusion	17
3.1.3	Projection	18
3.2	Complexité théorique en temps des algorithmes présentés	19
4	Analyse des résultats	20
5	Gestion du Projet	21
5.1	Répartition des roles	21
5.2	Diagramme de Gantt	21
5.3	Branches github, commits	22
5.4	Changements majeurs effectués en cours de projet	22
6	Bilan et Conclusions	23
6.1	Fonctionnalités réalisées présentent dans le cahier des charges	23
6.2	Fonctionnalités non-réalisées présentent dans le cahier des charges	23
6.3	Perspectives	23

1 Présentation du Sujet

Ce projet de programmation 2 a été réalisé dans le cadre de la licence informatique L3 de la faculté des sciences de Montpellier. L'objectif de ce projet est de simuler un fluide de manière interactive en 2D en temps réel. Pour cela, nous avons choisi d'implémenter la méthode de Jos Stam "Stable Fluids", qui permet de simuler un fluide de manière stable et rapide. Nous avons également implémenté une interface graphique pour visualiser en temps réel l'évolution du fluide et pour interagir avec celui-ci. Cette interaction est possible grâce à la création d'obstacles clicables et la possibilité de modifier durant la simulation les différents paramètres du fluide.

Étant donné notre parcours en double licence mathématique et informatique, ce projet est bien adapté à nos compétences. En effet, nous avons pu rapidement comprendre les concepts de base de la simulation, ce qui nous a permis d'implémenter la méthode de Jos Stam "Stable Fluids" .

L'implémentation d'une interface graphique pour interagir avec le fluide et observer son comportement nous a fortement intéressé. De plus, au travers ce projet, nous avons découvert le langage C++ et la bibliothèque OpenGL, qui sont des outils essentiels dans de nombreux domaines tels que la simulation de fluides, les jeux vidéos, la météorologie ou l'environnement.

Principalement trois familles de méthodes existent pour implémenter une simulation de fluide : les méthodes eulériennes, les méthodes lagrangiennes, et les méthodes semi-lagrangiennes, qui sont des hybrides des deux précédentes. Les méthodes eulériennes consistent à représenter le fluide dans une grille, où l'on stocke les différentes propriétés du fluide, telles que la densité, la chaleur, la vitesse, etc. Chaque cellule représente une portion du fluide, encodant les propriétés du fluide dans une région en un instant t .

Les méthodes lagrangiennes consistent, quant à elles, à représenter le fluide à l'aide de nombreuses particules, représentant les gouttes ou molécules du fluides. L'ensemble de ces particules déterminent les propriétés du fluide.

Pour l'implémentation de ce projet, nous avons choisi la méthode eulérienne, qui est plus simple à implémenter et moins coûteuse en complexité. La méthode lagrangienne peut rapidement exploser en temps de calcul en fonction du nombre de particules.

En effet, dans une simulation lagrangienne, chaque particule doit interagir avec toutes les autres particules, ce qui implique une complexité en $\mathcal{O}(n^2)$ (où n est le nombre de particules). Dans une simulation eulérienne, les calculs sont effectués sur une grille, ce qui permet de réduire la complexité à $\mathcal{O}(n)$ (où n est le nombre de cellules).

Le cahier des charges du projet, réalisé en décembre 2025, fixait le cadre de développement, et définissait les objectifs, les contraintes et les fonctionnalités (minimales) à implémenter. Vous trouverez à la suite une synthèse de celui-ci.

- Le projet consistait à développer un rendu interactif permettant la visualisation et la manipulation en temps réel d'une simulation de fluide en 2D. Comme précisé lors de la réunion avec notre responsable de projet, la simulation devait être interactive et comporter une interface graphique. Nous avons fait le choix de développer ce projet en C++ avec un affichage en OpenGL.
- L'objectif principal était d'obtenir une zone de rendu avec, d'un côté la simulation, et de l'autre une fenêtre de gestion des paramètres. Lors de la première réunion nous avons également déterminé le périmètre du projet. Nous avons décidé que la simulation reposerait sur une approche Eulerienne du fluide dans une grille 2D de taille 100×100 .
- Nous avons également pensé aux fonctionnalités que nous pourrions rajouter comme la connexion bluetooth à un téléphone permettant de gérer la simulation ou encore l'ajout de musique en fonction des forces du fluide. Enfin, nous avons imaginé une organisation modulaire (calcul du fluide, interactions, affichage), permettant une répartition optimale des tâches et un découpage plus propre de notre code.

2 Développements Logiciel : Conception, Modélisation, Implémentation

2.1 Technologies utilisées

- langage de programmation : C++ (justification pour l'optimisation, etc)

- OpenGL (affichage : api graphique)
- ImGui (facile d'utilisation)

Ça, ça devait changer..!

Pour notre projet, nous hésitions entre du C++ et du Python ; nous avons opté pour du C++ pour un typage plus fort et une meilleure gestion de ce qu'il se passe en mémoire et en interne plus généralement.

"De nombreux langages de programmation existent, néanmoins nous nous sommes orientés sur le langage C++ : en plus de proposer une importante puissance de calcul nécessaire pour la simulation de fluide, ce langage fortement typé nous oblige à collaborer avec plus de rigueur, ... bla bla bla ... "

~~À partir de la nous avons deux options concernant l'API graphique : SFML et OpenGL~~ Il existe plusieurs API graphiques pour l'affichage à l'écran, notamment OpenGL, Vulkan, SDL, SFML, etc. Nous avons ici choisi OpenGL afin de pouvoir déléguer au GPU et parce que nous avons déjà quelques bases dans le langage (grâce à l'excellent cours Learn OpenGL de Joey De Vries [4]).

~~Une fois en C++ sur OpenGL,~~ nous avons établi des prémices d'interactivité via le terminal, avant de basculer sur ImGui (~~voir le Github polyvalent de Oernut~~) [3], qui s'impose comme une référence en terme de panneau de contrôle interactif en OpenGL.

2.2 Développements logiciel réalisés

Dans cette partie nous allons nous intéresser à l'architecture de notre projet.

Le diagramme UML (Figure 1) suivant décrit brièvement l'organisation du projet que nous décrirons en détails par la suite. Nous pouvons clairement distinguer les principaux modules :

- Données : module qui regroupe les structures représentant la grille, les cellules et les particules (nous vons également inclu la classe Simulation pour des raisons de clarté) ;
- Fluid Solver : qui implémente les étapes de la simulation ;
- Zone de rendu : ce module permet l'affichage en OpenGL ;
- Fenêtre de paramètres : interface ImGui ;
- Interaction utilisateur : ce module traite les actions du clavier ou de la souris ;
- Gestion des obstacles : il a pour but de réunir ce qui touche aux obstacles, la création et le déplacement de ceux-ci.

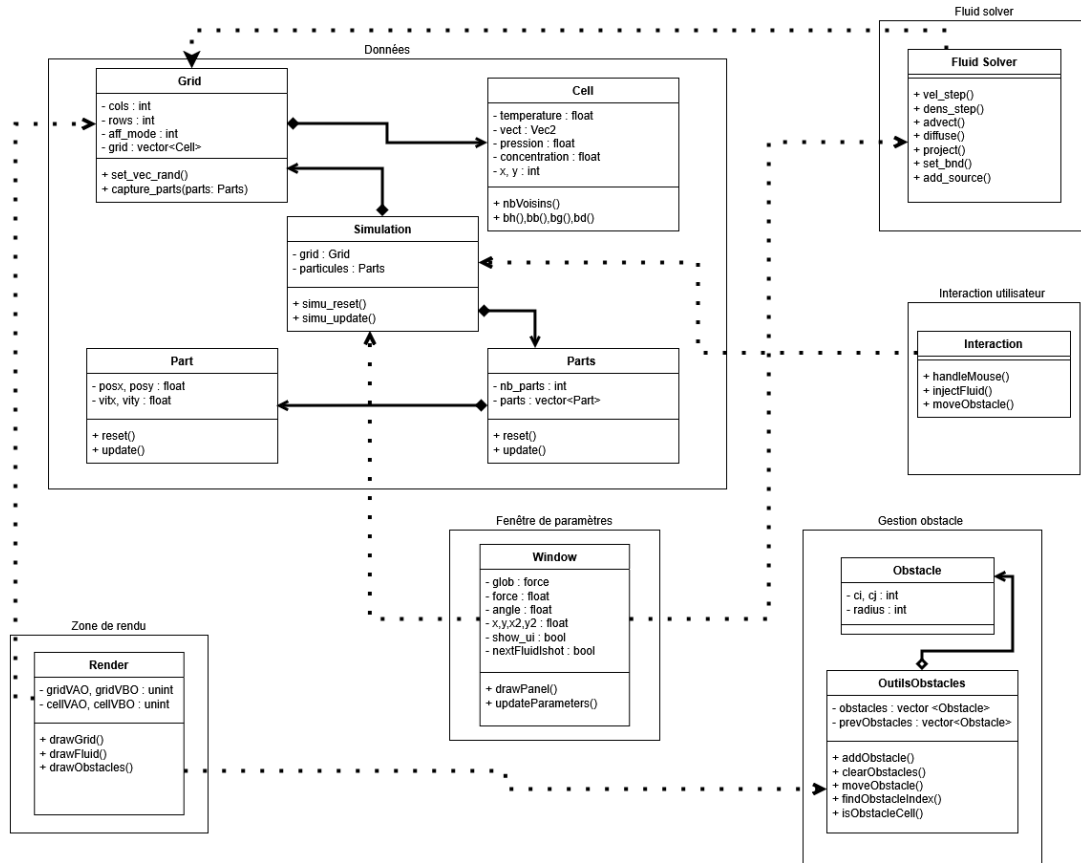


FIGURE 1 – Diagramme UML de notre projet

Ce diagramme permet de visualiser les dépendances entre modules et la structure de la simulation, les flèches indiquent des agrégations si le losange est vide, ou des compositions si le losange est plein, les pointillés décrivent des dépendances.

Nous allons maintenant présenter les différents composants de l’affichage et de l’interface de notre projet. L’interface finale repose sur plusieurs éléments, l’affichage d’une grille de chaleur, la visualisation du fluide, un panneau de paramètres et la gestion des obstacles. Ces composants constituent l’ensemble des fonctionnalités mis en oeuvre durant le projet.

2.2.1 Affichage d’une grille de chaleur

Nous avons mis en place un affichage sous forme de grille de chaleur (Figure 2). Pour mettre en place cet affichage, chaque cellule de la grille est associée à une valeur, dans ce cas, cette valeur représente une température. Nous utilisons différentes teintes de rouge afin de représenter visuellement les variations de température : une cellule très chaude apparaît plus rouge, tandis qu’une cellule voisine un peu moins chaude aura une teinte moins rouge. Cet affichage permet de visualiser intuitivement l’évolution d’un champ scalaire.

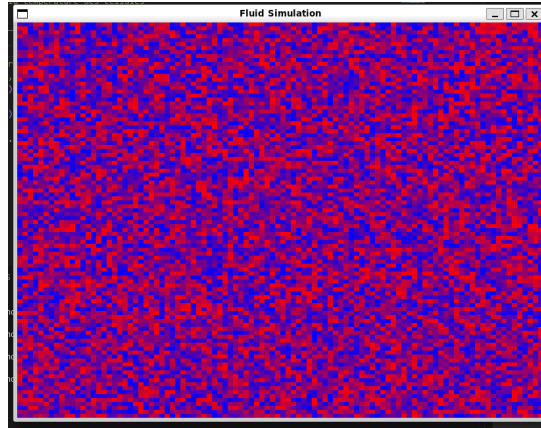


FIGURE 2 – Affichage d’une grille de chaleur. Le rouge représente les points les plus chauds et le bleu les plus froids.

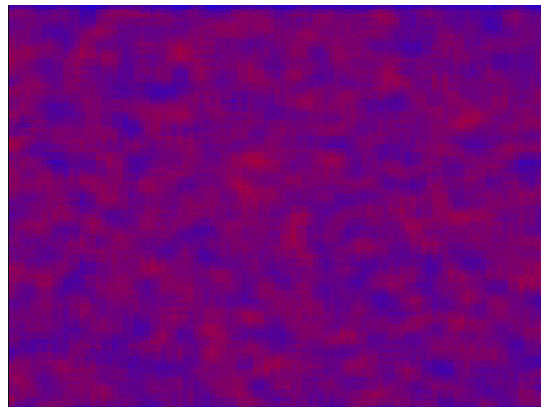


FIGURE 3 – Affichage d’une grille de chaleur évolution 2 au cours du temps.

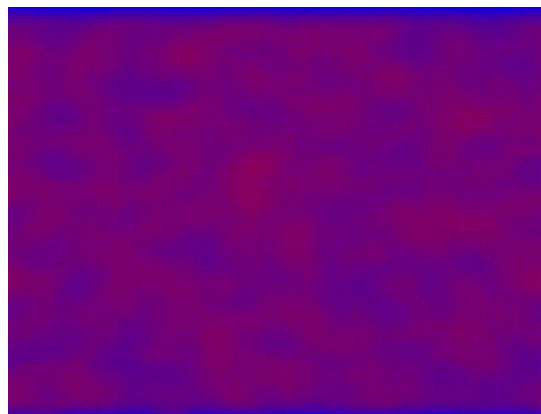


FIGURE 4 – Affichage d’une grille de chaleur, évolution 3 au cours du temps

L’évolution de la chaleur sur la grille suit ce principe : à chaque itération, la chaleur se propage progressivement vers les cellules voisines. Concrètement, une zone chaude a tendance à s’étaler autour d’elle. les taches rouges se déplacent, s’étirent et se mélangent au fil du temps, ce qui permet de visualiser facilement comment la température se diffuse dans le fluide.

Nous avons une simulation de fluide Eulérienne, qui fonctionne sur le principe d’une grille dont

chaque cellule possède ses paramètres locaux. Nous avons donc implémenté notre grille avec un simple `std::vector` C++ de structure `Cell` avec chaque paramètre comme attribut.

`std::vector` = tableau unidimensionnel (1D) -> car plus rapide sur le CPU pour faire des calculs.

Ajouter une image du diagramme UML avec seulement "Simulation", "Grid" et "Cell", puis la référencer dans le texte.

Cette grille possède ses propres méthodes (`reset`, `update`, *etc...*). Nous l'avons définie à la base de notre arborescence d'inclusion de fichiers afin d'en faire un Singleton : structure unique de référence accessible depuis n'importe quel module du projet.

Attention, ce n'est pas le cas dans votre diagramme UML. Vous pouvez dire que vous l'avez divisée en deux parties : "Grid" et "Simulation".

2.2.2 Simulation du fluide

Section à merger avec celle au-dessus ?

La simulation du fluide repose sur la méthode Jos Stam *Real-Time Fluid Dynamics for Games* [2] qui nous a permis d'avoir un rendu dynamique en 2D. Le fluide est représenté sur une grille Eulerienne (voir partie 1). Dans cette grille, chaque cellule stocke des informations physiques comme la densité ou la vitesse sous forme de scalaire (`float`) ou de vecteur (paire de `float`). A chaque itération de la simulation, ces données sont mis à jour grâce aux méthodes physiques comme l'advection, la diffusion ou la projection que nous présenterons en détails dans la section 3 . La zone de rendu affiche donc la densité du fluide en temps réel.

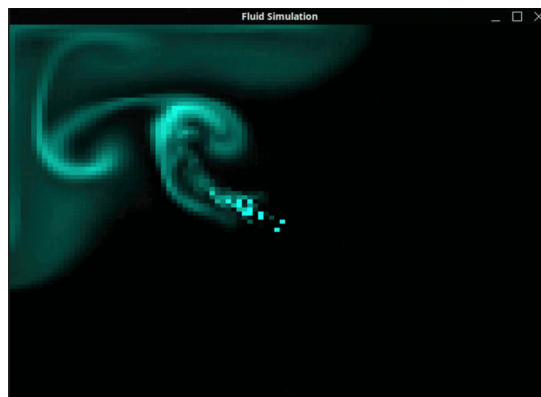


FIGURE 5 – Deuxieme version : affichage d'un fluide.

2.2.3 Mode point/ligne

Section "Interface Utilisateur" ?

Le choix est donné à l'utilisateur de la forme de la source du fluide. L'utilisateur peut injecter le fluide provenant soit d'un point unique, soit le long d'une ligne définie par deux coordonnées dans la grille soit quatre scalaires (voir figure ...).

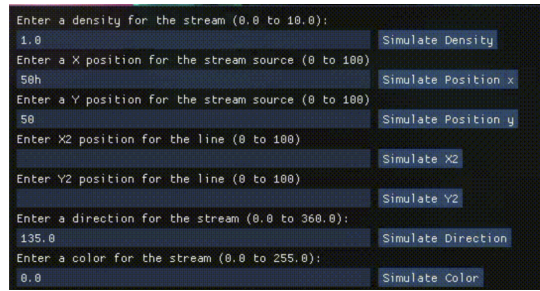


FIGURE 6 – Injection du fluide selon un segment

En mode point, la densité et les forces sont appliquées uniquement à la cellule correspondant à la position sélectionnée.

En mode ligne : nous déterminons simplement le nombre d'étapes nécessaires pour relier les deux points $(i_0, j_0), (i_1, j_1)$ en utilisant que le nombre d'étapes nécessaire est :

$step = \max(|i_1 - i_0|, |j_1 - j_0|)$. Lors ces étapes, nous parcourons, simplement toutes les cellules situées entre les deux positions sélectionnées, en suivant le segment qui les relie. Chaque cellule rencontrée reçoit la densité et la force, ce qui permet de créer une source linéaire.

2.2.4 Gestion de la chaleur

Un mode d'affichage thermique permet de visualiser le fluide en fonction de sa température. Pour implémenter la chaleur dans notre simulation, nous avons ajouté un paramètre "chaleur" à chaque cellule de la grille. Ce paramètre est représenté par une valeur numérique qui peut être positive (indiquant une source chaude) ou négative (indiquant une source froide).

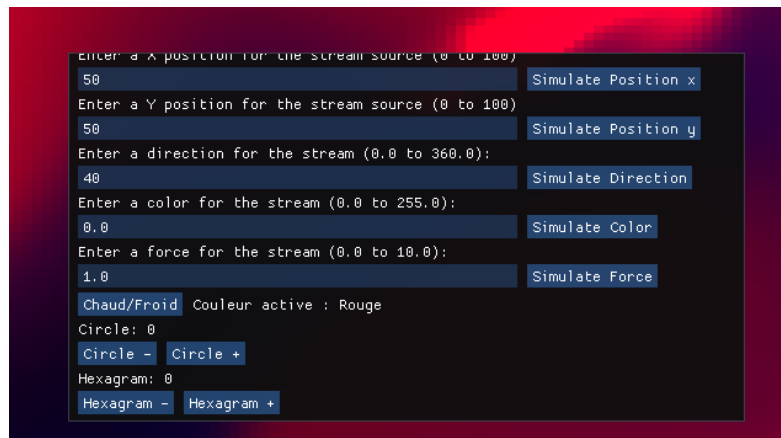
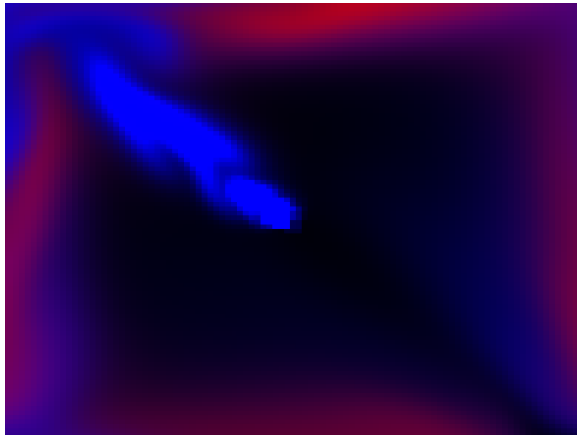
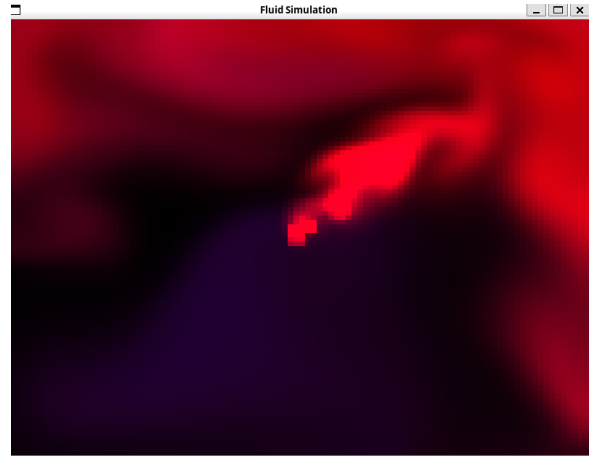


FIGURE 7 – Bouton Chaud/Froid

Dans l'interface graphique, nous avons ajouté un bouton "Chaud/Froid" qui permet à l'utilisateur de choisir si le fluide injecté est chaud ou froid.



(a) Affichage source froide



(b) Affichage source chaude

FIGURE 8 – Comparaison des sources thermiques

Quand on lance la simulation, le fluide est injecté par une source froide (bleue) par défaut (voir figure 8a). C'est grâce au bouton Chaud/Froid que l'on peut alterner et obtenir une source chaude (rouge) (voir figure 8b), on pourra aussi remarquer que la direction a été changée arbitrairement. On peut observer que sur la figure 8a, on a alterné déjà quelques fois entre les deux sources, ce qui explique la présence de quelques taches rouges et dégradés de violet.

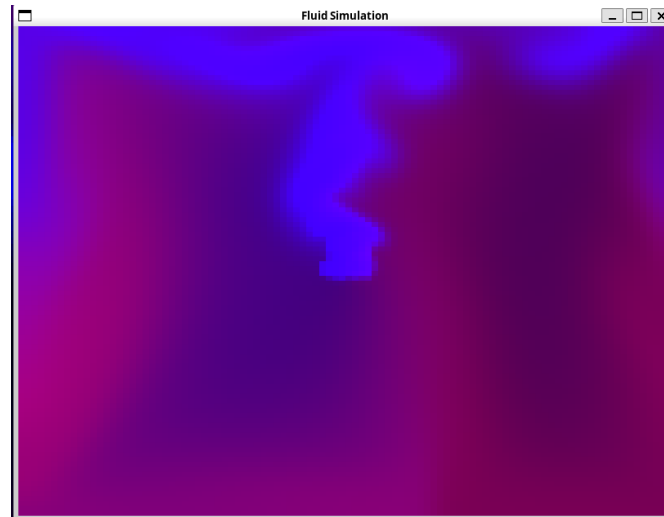


FIGURE 9 – représentation température

Dans la figure 9, on observe la simulation après de nombreuses itérations et on peut voir que le mélange est beaucoup plus homogène. On obtient comme un voile violet, symbolisant une température tiède assez uniforme, avec quelques zones plus chaudes (rouge) ou plus froides (bleu).

En termes d'implémentation, chaque cellule de la grille contient un paramètre *temperature* qui varie entre -1 (froid) et 1 (chaud). La propagation de la chaleur se fait par diffusion, c'est à dire que chaque cellule prend comme valeur la température moyenne de ses voisins à chaque itération. La chaleur ne peut pas d'échapper des bords. Chaque cellule se diffuse uniquement avec ses voisins à l'intérieur de la grille. Aux bords, elle se mélange avec moins de cellules voisines, ce qui peut entraîner une accumulation de froid ou de chaud le long des bords.

Ce mode d'affichage thermique peut être combiné avec les obstacles :

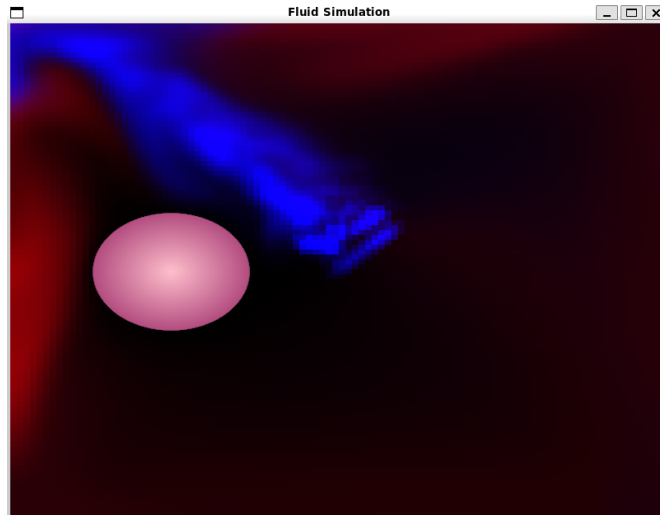


FIGURE 10 – chaleur + obstacles

On peut déplacer le cercle pour mélanger les différentes zones de température. Dans cette simulation, on considère que les obstacles sont neutres en terme de température, c'est à dire qu'ils n'ont pas d'effet de sur l'aspect thermique du fluide. Concrètement, dans l'implémentation, les obstacles sont comme des bords pour la chaleur : ils empêchent la chaleur de se diffuser à travers eux.

Lorsqu'on déplace un obstacle, il déplace également les zones de chaleur avec lui, ce qui peut créer des effets de mélange intéressants.

2.2.5 Gestion des obstacles

L'affichage possède également un système de gestion des obstacles. L'utilisateur peut activer ou désactiver les obstacles, changer leur forme ou en ajouter plusieurs (voir Figure 13). Chaque obstacle peut être déplacé grâce au curseur de la souris, cela nous permet d'observer son impact sur la simulation.

Attention, gardez le même style. Ici, je conseille de retirer le gras du texte.

Dans un premier temps, nous avons implémenté la création d'un unique obstacle, en modifiant notamment la fonction de gestion des bords afin qu'elle reconnaisse les obstacles. Nous pouvions également afficher ou non cet obstacle à l'aide d'un bouton on/off. Ajouter une image de ce bouton et ajouter une référence à la figure qui nous ramènera à l'utilisation ou non de la fonction lançant la création de l'obstacle.

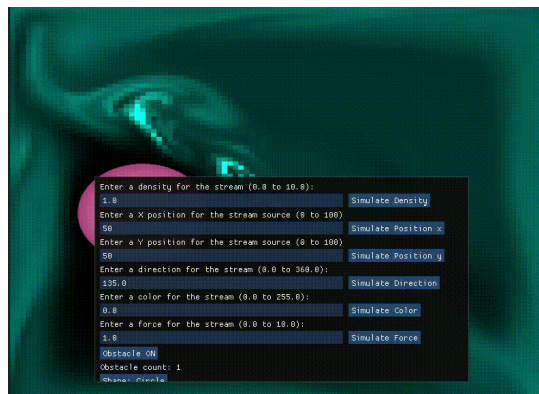


FIGURE 11 – Bouton on/off pour gérer l'affichage d'un obstacle.

On constate sur la figure 12 que l'obstacle agit sur le fluide et qu'il est activable ou non grâce au bouton on/off. Par la suite, nous avons rendu cet obstacle le plus interactif possible en offrant à l'utilisateur la possibilité de le déplacer à l'aide du curseur de la souris.

C'est quoi la fonction de gestion des bords ? Qu'est-ce que vous avez modifié dedans ? Au niveau programmation, ça demande quoi comme changements pour pouvoir déplacer un obstacle ?

Nous avons donc dans un premier temps adapter la fonction de gestion des bords. Cette fonction est une partie essentielle de la simulation, car elle permet de définir le comportement du fluide aux limites du domaine. Elle sert notamment à empêcher le fluide de sortir de la zone simulée et à gérer les interactions avec les parois, en imposant certaines conditions sur les vitesses (par exemple réflexion ou annulation de la vitesse).

Dans notre projet, cette fonction a été modifiée afin de prendre en compte la présence d'obstacles à l'intérieur du fluide. A l'origine, seul les bords du domaine était considérée. Nous avons donc adapté cette fonction pour traiter également les cellules appartenant aux obstacles . Cela implique de détecter si une cellule se situe à l'intérieur d'un obstacle ou à proximité de son bord, puis de modifier le comportement du fluide en fonction de ses informations. (par exemple en annulant la vitesse ou en appliquant une réflexion).

Au niveau programmation, permettre le déplacement d'un obstacle nécessite plusieurs modifications. Tout d'abord, il faut stocker les obstacles dans une structure de données (ici une liste) contenant leurs paramètres (coordonnées, rayon, forme). Ensuite, il est nécessaire de mettre à jour leur position à chaque interaction de l'utilisateur (par exemple le déplacement de la souris). Enfin, la simulation doit être recalculée en tenant compte de ces nouvelles positions, ce qui implique de mettre à jour les calculs de collision et les conditions aux bords en temps réel (il a notamment fallut utiliser Imgui afin de recuperer les coordonnées des cellules).

De plus, afin de conserver un comportement physique cohérent, nous avons introduit le calcul de la vitesse des obstacles en comparant leur position actuelle à celle de la l'itération précédente. Cette information est utilisée pour influencer le fluide au niveau des obstacles, ce qui permet d'obtenir des effets plus réalistes lors de leur déplacement.

Nous avons ensuite enrichi notre simulation en ajoutant la possibilité de gérer plusieurs obstacles simultanément, ainsi que de choisir leur forme parmi un cercle, un cœur ou une étoile. Pour cela, nous avons implémenter un curseur comptant le nombre d'obstacle par forme ainsi qu'un bouton +/- sur chaque forme possible afin de pouvoir ajouter ou supprimer les différents obstacles. Les obstacles sont stockés dans une unique **liste** contenant leurs coordonnées (x, y), rayon et forme, ce qui permet de les manipuler facilement (ajout, suppression, sélection et déplacement) tout en créant un ordre de priorité

liste de quoi ?

entre les différents obstacles (par exemple si 2 obstacle sont superposés, le premier a être déplacé sera le premier obstacle affiché et le premier obstacle supprimé sera le dernier obstacle créé). L'utilisateur peut ainsi ajuster dynamiquement le nombre d'obstacles via une interface dédiée.

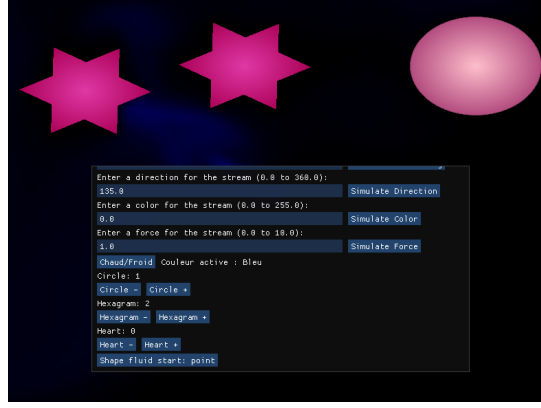


FIGURE 12 – Bouton + ou - pour gérer l'affichage des obstacles.

on constate bien que sur la figure 12 le nombre d'obstacles peut être ajusté dynamiquement grâce à ces boutons + ou - et de plus, il y a un curseur comptant le nombre d'obstacle présent. Afin de représenter précisément les différentes formes et leurs interactions avec le fluide, nous avons implémenté des fonctions mathématiques permettant, pour chaque point (x, y) , de déterminer sa position par rapport à un obstacle (intérieur, extérieur ou bord). Par exemple, dans le cas d'un cercle de centre (cx, cy) et de rayon r , la distance au bord est donnée par :

$$d(x, y) = \sqrt{(x - cx)^2 + (y - cy)^2} - r$$

Ainsi, si $d < 0$, le point est à l'intérieur du disque (donc le fluide ne peut pas y pénétrer), si $d = 0$, il est sur le bord (donc le fluide est en contact avec l'obstacle), et si $d > 0$, il est à l'extérieur (donc le fluide peut y pénétrer).

Pour des formes plus complexes comme le cœur, nous utilisons une forme paramétrique définie à l'aide d'un paramètre $t \in [0, 2\pi]$:

$$x(t) = 16 \sin^3(t)$$

$$y(t) = 13 \cos(t) - 5 \cos(2t) - 2 \cos(3t) - \cos(4t)$$

[5]. Ces coordonnées sont ensuite normalisées, redimensionnées et translatées afin de positionner correctement le cœur dans la simulation. Cette approche permet d'obtenir une forme réaliste et cohérente.

De même, l'étoile est conçu à partir de deux triangles équilatéraux centré en un meme point mais orientés différemment.

???

Enfin, pour améliorer le plus possible le réalisme de la simulation, nous avons mis en place une gestion des interactions entre le fluide et les obstacles. En utilisant en particulier, la fonction `obstacleNormal` qui permet de calculer la normale au bord de chaque obstacle, c'est-à-dire la direction perpendiculaire à sa surface.

Dans notre simulation, la normale est utilisée principalement lors des interactions entre le fluide et les bords des obstacles. Lorsque le fluide arrive au contact d'un obstacle, la normale permet de déterminer comment la vitesse du fluide doit être modifiée, notamment pour simuler un phénomène de réflexion.

Plus précisément, ce qui est réfléchi dans la simulation est la vitesse du fluide. Lorsqu'une cellule de fluide arrive au voisinage d'un obstacle, sa vitesse est modifiée en fonction de la normale afin de produire un effet de rebond réaliste.

La normale est calculée à partir de la fonction donnant la distance au bord de l'obstacle (ici la fonction `obstacleSignedDistance`, elle renvoie la distance entre le centre et le bord de l'obstacle). On approxime le gradient de cette fonction à l'aide de différences finies :

$$n_x \approx \frac{d(x + \varepsilon, y) - d(x, y)}{\varepsilon}, \quad n_y \approx \frac{d(x, y + \varepsilon) - d(x, y)}{\varepsilon}$$

où $d(x, y)$ représente la distance au bord de l'obstacle et ε est une petite valeur. Le vecteur obtenu est ensuite normalisé :

$$\vec{n} = \frac{(n_x, n_y)}{\sqrt{n_x^2 + n_y^2}}$$

Cette méthode permet d'obtenir une normale précise même pour des formes complexes comme le cœur ou l'étoile.

Si cette réflexion n'était pas mise en place, le fluide traverserait les obstacles sans interaction réaliste. Cela entraînerait des comportements inadaptés, comme la disparition du fluide à l'intérieur des objets ou des fuites à travers les bords, rendant la simulation incohérente. Il s'agit donc d'une information essentiel pour garantir une interaction cohérente entre le fluide et les obstacles, et pour maintenir le réalisme de la simulation.

La normale sert à quoi ? Quand est-ce que vous l'utilisez ? Qu'est ce qui est réfléchi dans la simulation ? Qu'est-ce qui se passerait si vous ne réfléchissiez pas les choses ?

La figure 13 illustre l'ajout de plusieurs obstacles dans la simulation (ici deux cercles, deux cœurs et une étoile à 6 branches) :

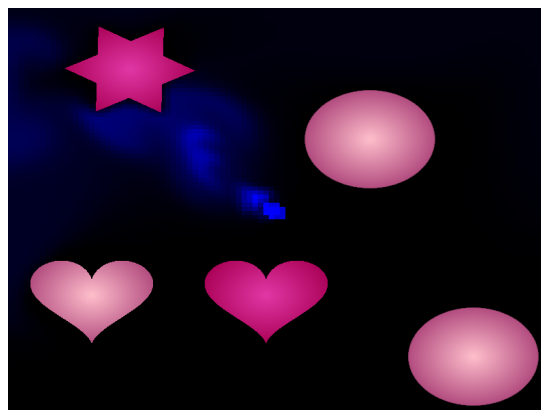


FIGURE 13 – Ajout de plusieurs obstacles.

2.2.6 Modules en cours de développement

Nous avons envisagé plusieurs fonctionnalités dans notre projet que nous n'avons pas pu finaliser par manque de temps.

Petit listing des 3 sous-modules

Interface utilisateur sur smartphone Parmi elles, nous avons une interactivité avec la simulation non pas avec IMGUI depuis la fenêtre OpenGL mais avec une application depuis une machine externe. Une application de test et une gestion de port à des fins de transmissions de données ont été développées. Malheureusement sans qu'une connexion ne puisse être établie, et ceux depuis plusieurs téléphones avec l'application d'OS différentes couplés avec plusieurs ordinateurs sous Linux et Windows.

Divise tes phrases, elles sont trop longues.

Semi-Lagrangien Pour enrichir notre simulation, nous souhaitons implémenter une méthode semi-lagrangienne. Le semi-Lagrangien étant une méthode de simulation hybride entre l'Eulérien (utilisation de grilles à paramètres) et du Lagrangien (utilisation de particules au comportement régi par des lois physiques) nous voulions utiliser notre grille déjà existante (mentionnée dans la partie [??Voir dans un commentaire au-dessus comment se référer à une autre section](#)) afin d'y capturer notre simulation

Lagrangienne de quelques particules. Une revisite de notre structure fût effective, nous avons redéfini notre structure-Singleton (référence, accessible de partout) en Structure Simulation : qui contient une structure grille, et notre nouvelle structure particules qui est ~~grossièrement~~ **principalement** un vecteur de structure particule, relativement analogue à la structure Cell (cellules de la grille). Référence à l’UML, ou ajout d’une figure avant/après de votre UML.

Calcul parallèle Une approche de parallélisme a été envisagée ; nous avons installé OpenMP pour faire donc du *Multi-Processing* sur OpenGL afin de pouvoir se permettre d’augmenter la résolution (taille de grille) de notre simulation, mais nous n’avons pu faire aboutir cette option par faute de temps.

2.3 Interface utilisateur

Notre panneau d’interaction avec la simulation permet de mettre la main sur de nombreuses variables de la simulation. Nous avons essayé de faire en sorte que chaque variable arbitraire devienne un curseur, une valeur à saisir, un interrupteur à enclencher afin de rendre notre simulation riche en personnalisation.

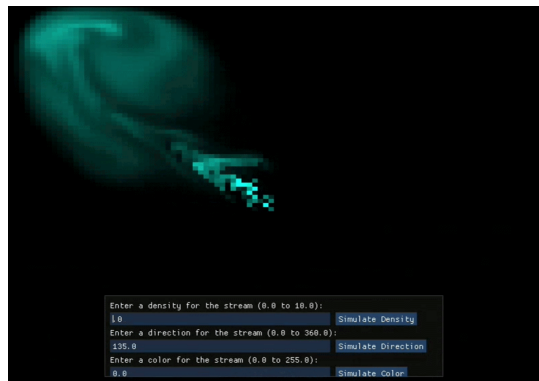


FIGURE 14 – Troisième version : ajout d’un panneau de paramètres.

Notre affichage comporte un panneau de paramètres développé avec ImGui. Ce panneau nous permet de modifier en temps réel, lors de la simulation, différents paramètres tels que la densité, la couleur ou encore la direction du fluide. Les valeurs saisies par l’utilisateur sont envoyées au module de simulation qui met à jour la grille. Ce panneau peut être affiché ou masqué en pressant la touche **h**.

Nous avons aussi des obstacles que nous pouvons ajouter, supprimer ou déplacer. On peut choisir leur nombre, leurs formes ainsi que leur position avec un simple clic ou déplacement de la souris.

2.4 Format des données d’entrée

- fluide stocké dans une grille
- grille composée de cellules
- cellules contiennent les paramètres de la simulation (densité, chaleur, vitesse)
- expliquer ce que représentent ces paramètres

Notre encodage des données se fait majoritairement via des ~~flottants~~ **scalaires**. Le fluide se décompose en plusieurs données physiques (densité, pression, viscosité, *etc...*) que l’on encode via des flottants et que l’on stock dans une structure Cell (cellule) dont on forme un vecteur d’ensemble dans une structure Grid (grille). Dans ce genre de structure on implémente des attributs-méthodes de tailles en ~~integers~~ **entiers**. On perfectionne notre diversité de simulation avec des conditions booléenne pour savoir si oui ou non, on affiche un obstacle, si oui ou non on a plusieurs points de fuites, *etc ...*

2.5 Statistiques

En termes de statistiques, notre projet est composé de structures et classes principales : Cellule, Grille, Obstacle, ImageMatrix, *etc*. Il est composé de 22 fichiers (cpp et h), et contient environ 2000 lignes de code. Le projet est organisé en modules logiques facilitant la maintenance.

Module	Fichier	Lignes
Physique	fluid_solver.cpp / .h	139 + 13
	fluides.cpp / .h	169 + 34
Obstacles	fluidRender.cpp / .h	394 + 44
Interface	main.cpp	916
Rendu	display.cpp / .h	424 + 66
Système	utils.cpp / .h	127 + 245
	matrix.cpp / .h	87 + 45
Bluetooth	interaction.cpp / .h	52 + 16
Total	15 fichiers	2771

TABLE 1 – Statistiques du projet : répartition des fichiers par module

3 Algorithmes et Structures de Données

3.1 Principaux algorithmes utilisés

Dans cette section, nous allons présenter et analyser l'un des algorithmes principaux du projet : `vel_step`. Pour cela, nous allons détailler 3 algorithmes essentiels : advection, diffusion et projection. `vel_step` est l'algorithme qui fait évoluer la vitesse du fluide à chaque itération de la simulation.

```

fonction vel_step(N, u, v, u_prev, v_prev, dt, visc)
    début
        ajouter_source(N, u, u_prev, dt)
        ajouter_source(N, v, v_prev, dt)

        échanger u et u_prev

        diffuse(N, 1, u, u_prev, visc, dt)
        Echanger v et v_prev
        diffuse(N, 2, v, v_prev, visc, dt)

        project(N, u, v, u_prev, v_prev)

        échanger u et u_prev, Echanger v et v_prev
        advect(N, 1, u, u_prev, u_prev, v_prev, dt)
        advect(N, 2, v, v_prev, u_prev, v_prev, dt)

        project(N, u, v, u_prev, v_prev)
    fin

```

L'algorithme `vel_step` prend en entrée les paramètres suivants :

- N : la taille de la grille (la simulation est définie sur une grille $N \times N$);
- u et v : les tableaux représentant les composantes horizontale et verticale du champ de vitesse du fluide;
- u_{prev} et v_{prev} : les tableaux contenant les valeurs du champ de vitesse au pas de temps précédent (ou des champs intermédiaires);
- dt : le pas de temps utilisé pour l'évolution temporelle de la simulation;
- visc : la viscosité du fluide, qui contrôle la diffusion du champ de vitesse.

3.1.1 Advection

L'advection est un processus physique qui décrit le transport d'une quantité (densité, chaleur ou vitesse) par le fluide. L'advection déplace l'information en suivant le mouvement du fluide. Selon le travail

de Jos Stam [1], nous utilisons l'approche *semi-lagrangienne* introduite. L'équation de l'advection est :

$$\frac{\partial x}{\partial t} + \mathbf{u} \cdot \nabla x = 0,$$

où x est la quantité transportée et $\mathbf{u} = (u, v)$ le champ de vitesse. L'idée est de déterminer, pour chaque cellule (i, j) , la position d'où provient le fluide arrivé à cet endroit au temps $t + \Delta t$. On remonte donc le long du champ de vitesse.

Comme (x', y') n'est généralement pas un point entier de la grille, on utilise une *interpolation bilinéaire* entre les quatre cellules voisines pour estimer la valeur transportée.

```

fonction ADVECT(N, b, d, d0, u, v, dt)

    dt0 ← dt × N

    pour i de 1 à N faire
        pour j de 1 à N faire

            # 1. Remonter le long du champ de vitesse
            x ← i - dt0 × u[i,j]
            y ← j - dt0 × v[i,j]

            # 2. Contraindre la position dans la grille
            si x < 0.5 alors x ← 0.5
            si x > N + 0.5 alors x ← N + 0.5
            si y < 0.5 alors y ← 0.5
            si y > N + 0.5 alors y ← N + 0.5

            i0 ← floor(x)
            i1 ← i0 + 1
            j0 ← floor(y)
            j1 ← j0 + 1

            # 3. Coefficients d'interpolation
            s1 ← x - i0
            s0 ← 1 - s1
            t1 ← y - j0
            t0 ← 1 - t1

            # 4. Interpolation bilinéaire
            d[i,j] ←
                s0 × ( t0 × d0[i0,j0] + t1 × d0[i0,j1] ) +
                s1 × ( t0 × d0[i1,j0] + t1 × d0[i1,j1] )

        fin pour
    fin pour

    appliquer_conditions_aux_limites(d)

fin fonction

```

L'algorithme d'advection prend en entrée plusieurs paramètres :

- N : la taille de la grille (la simulation est définie sur une grille $N \times N$) ;
- b : un paramètre indiquant le type de bord à appliquer (bord droit, gauche, haut, bas) ;
- d : le tableau dans lequel on écrit la nouvelle valeur advectée ;
- d_0 : le tableau contenant les valeurs de la quantité au pas de temps précédent ;
- u et v : les composantes horizontale et verticale du champ de vitesse ;

— dt : le pas de temps utilisé pour remonter le long du champ de vitesse.

Ces paramètres permettent à l'algorithme de déterminer d'où provient la quantité transportée, de calculer la nouvelle valeur par interpolation, puis d'appliquer les conditions de bord. En effet, si la matière arrive sur le bord droit, le mouvement du fluide n'est pas le même que le bord droit, gauche ou haut.

L'advection semi-lagrangienne peut être vue comme un déplacement de matière : au lieu de pousser la matière vers l'avant, on cherche d'où elle provient et on la pousse dans la direction continue. Visuellement, l'advection est responsable du mouvement du fluide.

3.1.2 Diffusion

Comme l'advection, la diffusion est un processus physique qui décrit la propagation d'une substance. Nous avons donc choisi de l'implémenter en pseudocode comme suit :

```
fonction DIFFUSE(N, x, x0, diff, dt)
    a ← dt × diff × N2

    pour k de 1 à 20 faire
        pour i de 1 à N faire
            pour j de 1 à N faire
                x[i,j] ← ( x0[i,j]
                    + a × ( x[i-1,j] + x[i+1,j]
                    + x[i,j-1] + x[i,j+1] ) )
                    / (1 + 4a)
            fin pour
        fin pour

        appliquer_conditions_aux_limites(x)
    fin pour

    retourner x

fin
```

L'algorithme de diffusion utilise l'équation aux dérivées partielles qui suivent :

$$\frac{\partial x}{\partial t} = D \nabla^2 x$$

où :

- x est la quantité diffusée (densité, température),
- D est le coefficient de diffusion,
- ∇^2 est le Laplacien.

En utilisant la discrétisation utilisée par Jos Stam on obtient :

$$x^{t+1} = x^t + \Delta t D \nabla^2 x^{t+1}$$

Ce choix est essentiel pour garantir la stabilité numérique, même pour de grands pas de temps. Sur une grille 2D, le Laplacien discret est approximé par :

$$\nabla^2 x_{i,j} \approx x_{i-1,j} + x_{i+1,j} + x_{i,j-1} + x_{i,j+1} - 4x_{i,j}$$

En remplaçant la formule de Jos Stam par l'approximation du Laplacien, on obtient :

$$x_{i,j}^{t+1} = \frac{x_{i,j}^0 + a (x_{i-1,j}^t + x_{i+1,j}^t + x_{i,j-1}^t + x_{i,j+1}^t)}{1 + 4a}$$

avec :

$$a = \Delta t \cdot D \cdot N^2$$

D'après Gauss-Seidel, l'équation précédente correspond à un système linéaire de la forme :

$$(I - a\Delta)x = x_0$$

Ce système est résolu par une méthode itérative de Gauss-Seidel. Elle consiste à :

- parcourir la grille plusieurs fois,
- mettre à jour chaque cellule immédiatement avec les nouvelles valeurs disponibles,
- répéter jusqu'à convergence.

Cette méthode accélère la convergence et permet une simulation stable et efficace. Ainsi, on peut interpréter la diffusion comme un processus qui rend les valeurs de plus en plus uniformes : Une zone où la quantité est très concentrée se diffuse progressivement vers les cellules voisines. Au fil des itérations, cela rend la répartition de plus en plus uniforme, ce qui est indispensable pour obtenir des simulations de fluides réalistes.

3.1.3 Projection

Après avoir appliqué les algorithmes d'advection et de diffusion, l'algorithme de projection implémente la décomposition de Hodge. C'est un algorithme qui permet de rendre le champ de vitesse du fluide incompressible et qui force la vitesse à conserver la masse. C'est une propriété importante des fluides réels.

Visuellement, la projection force l'écoulement à avoir de meilleures courbes, et évite les écoulements en ligne droite.

La décomposition de Hodge nous dit que tout champ de vecteur de vitesse est la somme d'un champ conservant la masse et d'un champ de gradient. L'idée est de rendre le champ de vitesse conservateur de masse lors de la dernière étape en soustrayant le champ de gradient du champ de vitesse actuel.

```
fonction PROJECT(N, u, v, p, div)
  h <- 1.0 / N

  # Étape 1
  pour chaque cellule (i, j) de la grille :
    div[i, j] = -0.5 * h * (u[i+1, j] - u[i-1, j] + v[i, j+1] - v[i, j-1])
    p[i, j] = 0
  fin pour
  appliquer_conditions_aux_limites(div)
  appliquer_conditions_aux_limites(p)

  # Étape 2
  répéter 20 fois :
    pour chaque cellule (i, j) de la grille :
      p[i, j] = (div[i, j] + p[i-1, j]
                + p[i+1, j] + p[i, j-1]
                + p[i, j+1]) / 4
    fin pour
    appliquer_conditions_aux_limites(p)
  fin répéter

  # Étape 3 :
  pour chaque cellule (i, j) de la grille :
    u[i, j] = u[i, j] - 0.5 * (p[i+1, j] - p[i-1, j]) / h
    v[i, j] = v[i, j] - 0.5 * (p[i, j+1] - p[i, j-1]) / h
  fin pour
  appliquer_conditions_aux_limites(u)
  appliquer_conditions_aux_limites(v)
fin fonction
```

L'algorithme de projection prend en entrée les paramètres suivants :

- N : la taille de la grille (la simulation est définie sur une grille $N \times N$) ;
- u et v : les composantes horizontale et verticale du champ de vitesse ;
- p : un tableau représentant le champ de pression, utilisé pour corriger le champ de vitesse ;
- div : un tableau contenant la divergence du champ de vitesse initial.

Cet algorithme est composé de 3 étapes :

Etape 1 : calcul de la divergence : pour chaque cellule de la grille, on évalue la divergence discrète

$$div_{i,j} = -\frac{h}{2}(u_{i+1,j} - u_{i-1,j} + v_{i,j+1} - v_{i,j-1}) \quad (1)$$

où $h = 1/N$ est la taille d'une cellule de la grille, et u et v sont les composantes de la vitesse du fluide.

Etape 2 : résolution de l'équation de Poisson

On résout itérativement l'équation $\nabla^2 p = div$ par 20 itérations de relaxation de Gauss-Seidel, où p est un champ de pression :

$$p_{i,j}^{(k+1)} = \frac{1}{4}(div_{i,j} + p_{i+1,j}^{(k)} + p_{i-1,j}^{(k)} + p_{i,j+1}^{(k)} + p_{i,j-1}^{(k)} - div_{i,j}) \quad (2)$$

On réutilise exactement le code de la diffusion pour assurer la cohérence du code et éviter les redondances.

Etape 3 : correction du champ de vitesse

pour chaque cellule de la grille, on corrige la vitesse en soustrayant le gradient du champ de pression :

$$u'_{i,j} = u_{i,j} - \frac{1}{2h}(p_{i+1,j} - p_{i-1,j}) \quad (3)$$

$$v'_{i,j} = v_{i,j} - \frac{1}{2h}(p_{i,j+1} - p_{i,j-1}) \quad (4)$$

3.2 Complexité théorique en temps des algorithmes présentés

Les algorithmes d'advection, de diffusion et de projection nous permettent de définir l'algorithme `vel_step`. Sa complexité en temps est déterminée par les 3 algorithmes principaux qui la composent :

Dans cette section, nous notons N le nombre de cellules totales de la grille.

Advection. L'algorithme d'advection met à jour la valeur de la densité ou la vitesse pour chaque cellule de la grille. L'algorithme parcourt donc l'ensemble des N cellules de la grille à l'aide d'une boucle *for*.

Pour chaque cellule, l'algorithme

- détermine la position d'origine du fluide
- effectue une interpolation
- met à jour la valeur de la densité ou vitesse

Toutes ces opérations sont réalisées en temps constant ($\mathcal{O}(1)$) pour chaque cellule. Comme l'algorithme parcourt les N cellules de la grille, le coût total est $\mathcal{O}(N)$.

A la fin de l'algorithme, il y a un appel à la fonction `set_bnd` qui parcourt elle aussi les N cellules et a une complexité $\mathcal{O}(N)$.

La complexité finale de l'algorithme *advection* est donc :

$$\mathcal{O}(N) + \mathcal{O}(N) = \mathcal{O}(N)$$

.

Diffusion. L'algorithme parcourt l'ensemble des N cellules de la grille à chaque itération, ce qui représente un coût de $\mathcal{O}(N)$. Comme le nombre d'itérations est constant (20), la complexité totale est en $\mathcal{O}(N)$.

Projection. L'algorithme de projection est composé de 3 étapes :

- Etape 1 : calcul de la divergence : N opérations
- Etape 2 : résolution de l'équation de Poisson : $20 \times N$ opérations
- Etape 3 : correction du champ de vitesse : N opérations

Ainsi, la complexité totale de l'algorithme de projection est en $\mathcal{O}(N)$.

Ce qui nous donne une complexité totale de $\mathcal{O}(N + N + N) = \mathcal{O}(N)$ pour l'algorithme `vel_step`.

4 Analyse des résultats

- eulerien vs lagrangien (uniquement eulerien a été implémenté)
- analyser les performances visuelles en fonction du nombre d'obstacles, densité, etc (mettre des screenshot)

On peut voir l'évolution de notre projet a travers son evolution visuelle. mais egalemt c'est performance en fonction de différents paramètres tels que la densité du fluide ou le nombre d'obstacles.

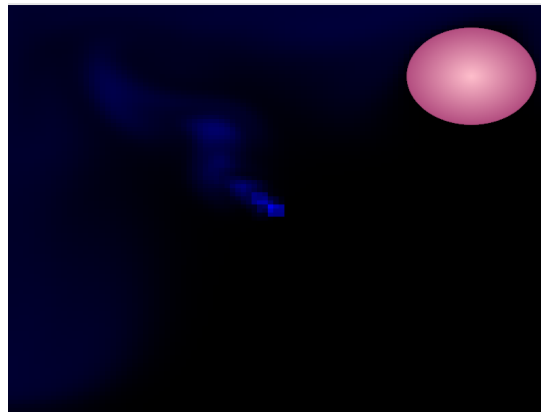


FIGURE 15 – Simulation avec un obstacle et toute les propriete du fluides non modifiées

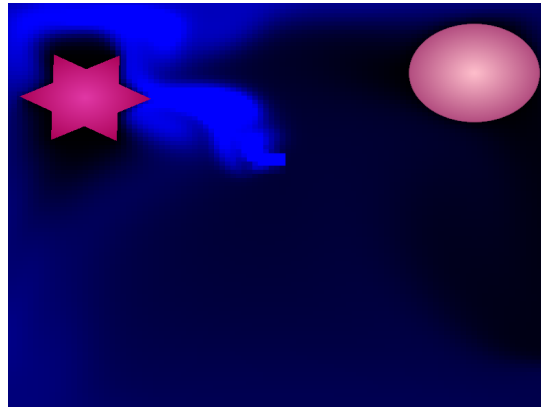


FIGURE 16 – Simulation avec 2 obstacles et une densité plus élevée(5)

On constate une meilleur visibilite des obstacle et du fluide ainsi que sa direction.

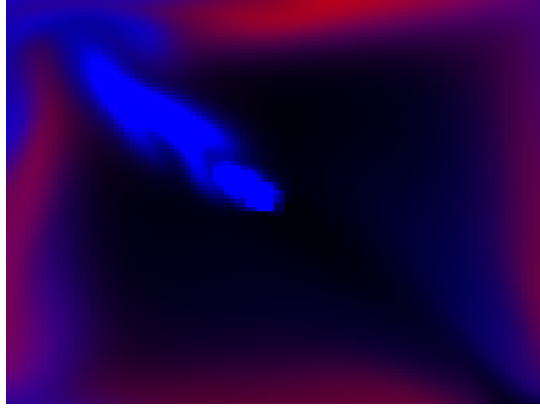


FIGURE 17 – Simulation avec alternance entre le rouge et le bleu pour représenter la chaleur du fluide

On remarque que le fluide sortant se mélange assez facilement en créant des dégradés de rouge et de bleu.

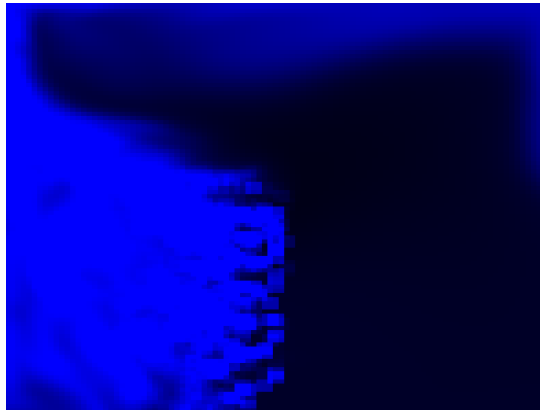


FIGURE 18 – Simulation avec sortie du fluide en ligne horizontale

On constate un fort fluide remplissant vite le cadre comparé au point de l'image au dessus.

C'est un peu mou de description tout ça...

5 Gestion du Projet

-(1-2 pages)

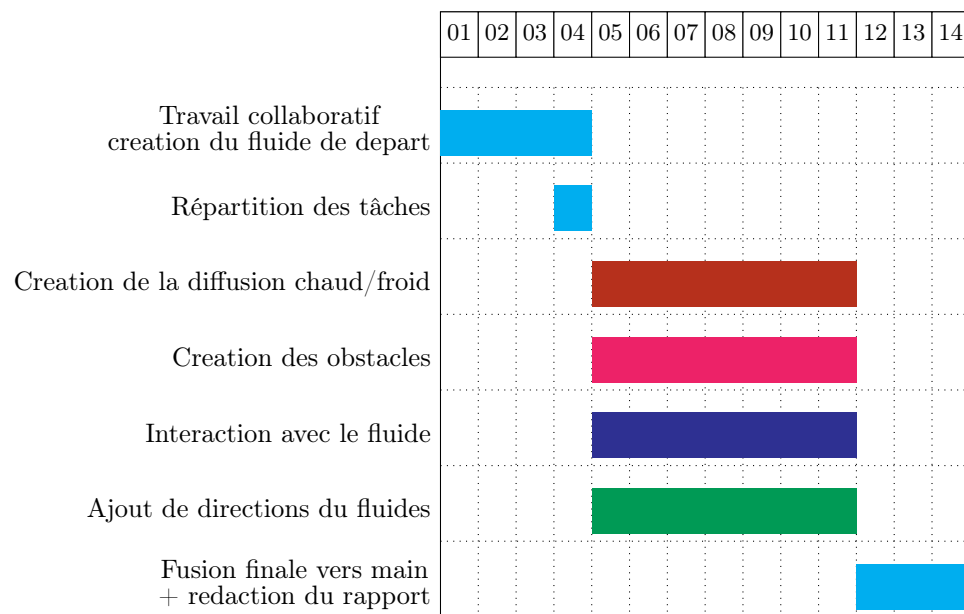
5.1 Répartition des rôles

Durant le projet, nous avons dans un premier temps travaillé ensemble pour la mise en place de la grille de départ et la création du fluide ainsi que pour l'affichage de celui-ci. Une fois cette partie terminée, nous avons ensuite décidé de nous répartir les tâches pour être plus efficace. Ainsi, Gaëlle a permis de déplacer le fluide dans diverses directions et à partir de divers points, Satine a travaillé sur les obstacles, Axel a généré le code ImGui afin de pouvoir modifier les différents paramètres du fluide (densité, couleur,...) et le Bluetooth, et Saurane a travaillé sur la diffusion de la chaleur.

5.2 Diagramme de Gantt

Nous avons réalisé un diagramme de Gantt pour planifier les différentes étapes du projet.

Organisation du travail - Diagramme de Gantt



Légende :

- ■ Gaëlle
- ■ Satine
- ■ Axel
- ■ Saurane
- ■ tout le monde (travail collaboratif sur la branche main)

5.3 Branches github, commits

Pour la mise en place d'un travail d'équipe optimale, nous avons utilisé GitHub. Lorsque nous travaillions ensemble, nous nous plaçons sur la branche main afin que chacun puisse accéder à ce que les autres ont fait facilement, puis lorsque nous avons choisi de nous répartir les tâches nous nous sommes tous mis à travailler sur des branches différentes (Gaëlle, Satine, Axel, Saurane) afin de pouvoir travailler et enregistrer notre travail sans risquer de causer des conflits ou d'enregistrer quelque chose qui ne fonctionne pas. Une fois notre programme complet nous le mettons alors sur le main afin que tout le monde puisse y accéder. Nous avons également veillé à faire des commit réguliers et descriptifs pour suivre l'évolution du projet et faciliter la collaboration.

5.4 Changements majeurs effectués en cours de projet

- passage de la grille de départ à la nouvelle grille (pourquoi?)

Nous avons au cours de notre projet dû réadapter notre structure, en effet, nous avons préparé une grille d'essai pour tester notre affichage, mais lorsqu'il a fallu confronter notre prototype aux besoins du papier *Stable Fluids* de Jos Stam, nous nous sommes rendu compte de besoins supplémentaires. Certains algorithmes du papier allaient avoir besoin d'échanger un certain paramètre de toutes les cellules et notre structure ne nous le permettaient pas. Nous avons donc revu l'organisation. Nouvelles méthodes, nouveaux attributs, *Setters/ Getters* plus efficace, etc...

- la façon d'implémenter les obstacles a été modifiée quand on est passé de l'affichage d'un obstacle à plusieurs, est-ce que ça a impacté les autres modules? pourquoi? comment?

Pour la création d'obstacle, il a fallu dans un premier temps réadapter la fonction qui délimite les bords du fluide afin que celui-ci prenne en compte les divers obstacles, ce qui a créé de nombreux problèmes (la disparition du fluide à travers l'obstacle ou la fuite du fluide à travers les bords),

ou encore des problèmes de dépendance entre les différents obstacles lorsque nous avons voulu ajouter plusieurs obstacles mobiles. De plus, l'implémentation d'obstacles ayant une autre forme que la boule (le premier obstacle programmer) pose de nombreux problèmes notamment dans la partie de délimitation du fluide ou encore dans la partie d'affichage.

Pour résoudre ces problèmes il a fallu revoir le code en changeant la façon de voir les obstacles mais également en les implémentant de manière différente. Ainsi, les obstacles ont été rangés dans une liste afin que chaque obstacle soit reconnaissable et manipulable facilement. Ils sont ainsi sélectionnés à partir de cette liste lorsqu'il faut les afficher, et c'est le dernier obstacle de la liste qui est supprimé lorsque l'on désire en retirer un. Cette organisation a également permis de mieux gérer les interactions entre plusieurs obstacles et de réduire les conflits lors de leurs déplacements.

Enfin, cette nouvelle structure a facilité l'ajout de formes différentes et leur gestion dans la simulation, en rendant le système plus flexible et plus stable dans le temps.

- implémentation de la fenêtre pour rentrer les paramètres

Un autre changement majeur a été l'implémentation d'une fenêtre interactive pour modifier en temps réel les paramètres de la simulation.

Initialement, les paramètres étaient prédéfinis en dur dans le code, ce qui limitait les possibilités d'expérimentation, mais également l'accessibilité du projet pour les utilisateurs qui n'ont pas de connaissances en programmation.

6 Bilan et Conclusions

6.1 Fonctionnalités réalisées présentent dans le cahier des charges

Parmi toutes les fonctionnalités que nous avons implémentées, certaines étaient déjà prévu au moment de l'écriture du cahier des charges.

- L'interface graphique composée de la fenêtre d'affichage du fluide : Cette interface graphique est bien présente dans notre projet et elle correspond presque à l'identique à ce que nous avons imaginé au départ.
- possibilité d'agir sur le fluide avec des curseurs : Nous avons en effet la possibilité de modifier les paramètres du fluide en temps réel, cependant, nous avons finalement opté pour des boutons et des champs de saisie plutôt que des curseurs. Ce choix a été assez naturel car il nous permet d'être plus précis dans le choix des paramètres, tels que la densité ou la vitesse.
- possibilité d'ajouter des obstacles : Cette fonctionnalité a fini par être l'une des principales de notre projet, et elle permet de rendre la simulation plus ludique et plus réaliste.

6.2 Fonctionnalités non-réalisées présentent dans le cahier des charges

Dans le cahier des charges initial, nous avons envisagé d'ajouter une fonctionnalité permettant de faire réagir la simulation à un signal sonore comme la musique. L'idée était d'utiliser la fréquence d'un flux audio (musique ou microphone) pour influencer certains paramètres du fluide, comme la densité injectée, la force appliquée ou encore la couleur. Cette approche aurait permis de créer une visualisation dynamique du fluide, synchronisée avec la musique.

Nous avons commencé à étudier plusieurs bibliothèques audio (notamment `SFML Audio` et `PortAudio`) afin d'extraire en temps réel des informations simples comme le volume instantané. Cependant, cette fonctionnalité nécessitait un traitement du signal plus avancé (FFT, filtrage, normalisation), ainsi qu'une intégration avec la boucle de simulation. Faute de temps, nous n'avons pas pu mettre en place cette idée.

Cette fonctionnalité reste néanmoins une piste intéressante pour enrichir la simulation.

6.3 Perspectives

- Semi - lagrangien
- autres fonctionnalités possibles avec les obstacles, etc

Comme mentionné dans la partie [??](#), les perspectives d'évolutions pour la suite du projet sont ces choses auxquelles nous avons songé, que nous avons essayé d'implémenter mais qui nous n'avons pas su faire aboutir. Parmi elles on peut compter :

Une meilleure interactivité : Nous reprendrions l'application et le code de transmission de données et de gestion des ports afin de rendre cette fonctionnalité opérationnelle.

Du Semi-Lagrangien : Nous achèverions notre implémentation du Semi-Lagrangien en intégrant la structure des particules à la mise à jour de la simulation et à l'affichage en Shader. On choisirait soit un bouton de choix entre l'Eulerien et le Semi-Lagrangien soit de superposer les deux, voir de les faire interagir, ce qui demanderait de faire des choix d'affichage en Shader et/ou de la gestion de contact.

Une meilleure résolution : Nous finirions d'implémenter et d'utiliser OpenMP pour paralléliser nos calculs de mise à jour de simulation afin de se permettre d'en faire plus. En réduisant le temps de calcul, on pourrait se permettre d'augmenter le nombre de cellule tout en gardant le même nombre d'images calculées par seconde, nous permettant d'ajouter une meilleure définition à la simulation (étant actuellement de 100 cellules par 100 cellules).

Références

- [1] Stam, J. (1999). Stable fluids. Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH 1999, 121–128. <https://doi.org/10.1145/311535.311548>
- [2] Stam, J. (2003). Real-Time Fluid Dynamics for Games. Proceedings of the Game Developer Conference, 18(11), 17.
- [3] <https://github.com/ocornut/imgui>
- [4] <https://learnopengl.com/>
- [5] <https://mathworld.wolfram.com/HeartCurve.html>